

Everyone Builds Everything

A rotation design that puts all five engineers through all five technical domains inside the same 25 September–8 November window — without the project losing the continuity it needs to ship.

5 DOMAINS EACH	4 ROTATION ROUNDS	20 HANDOVERS	44% TIME OUTSIDE HOME	~11% THROUGHPUT COST	60 h SCOPE YOU CUT
--------------------------------------	---	------------------------------------	---	--	--

The problem with just rotating people

Cross-training is the right ambition. It is also the thing most likely to sink a 21-day project, and it fails in two specific ways that are worth naming before designing around them.

Failure one — nobody owns the shape

If authorship rotates and nothing else does, every domain drifts. The schema grows three naming conventions. The feature builder gets a fourth signature. Nobody notices that a rule was duplicated, because the person who wrote it in week 3 was somewhere else in week 5. By week 6 the integration cost exceeds the remaining capacity.

Failure two — the viva gets worse, not better

The individual viva asks two questions: *explain this system* and *what did you build and why*. Broad rotation improves the first answer and can destroy the second. Five people who each touched everything shallowly and decided nothing is a worse outcome than five specialists, and examiners notice it immediately.

THE DESIGN RESPONSE

Separate the two things that lane ownership bundles together. **Stewardship** — accountability for a domain's integrity across all seven weekends — does *not* rotate. **Authorship** — who actually writes the code this weekend — does. Everyone keeps one domain they can defend in depth, and everyone still builds in all five.

Three mechanisms, in order of importance

1. **Vertical slices instead of horizontal tasks.** The single biggest cross-training lever, and it costs nothing. Assign “*creator rate card, end to end*” — migration, model, repository, service, endpoint, screen, test — rather than “*build the rate card API*”. One person walks the whole stack for one feature. A slice is finishable inside one weekend, which is exactly what a three-day cadence needs.
2. **Pair-and-swap on Saturdays.** Each weekend every person spends their deepest block in a domain they do not steward, with the steward pairing for the first two hours and reviewing on Sunday. Four rounds put everyone through all four other domains.
3. **Teaching artifacts.** Every visit ends with ten lines in `docs/domains/<domain>/field-notes.md`: what I built, what surprised me, where the traps are. This is how knowledge outlives the rotation — and it is also, almost for free, the individual contribution log the viva needs.

WHY THE LAYER CONTRACT MAKES THIS SAFE

Rotation only works if a newcomer cannot break something they do not understand. The layer contract in Part 3 of the Build Manual is what provides that: routers cannot query, services cannot write SQL, ML modules cannot see a `Session`. A visitor writing their first endpoint in an unfamiliar domain is boxed in by the architecture rather than by their own caution. **Enforce the contract strictly from weekend 1 and rotation becomes cheap; let it slide and rotation becomes dangerous.**

The five domains and their stewards

Stewardship is fixed for all seven weekends. A steward decides; a visitor implements. That one rule is what keeps five pairs of hands from producing five architectures.

DOMAIN	STEWARD	COVERS	STEWARD'S STANDING JOB
D1 PLATFORM	Harshit	Migrations, auth, deps, schemas, services, repositories, error shape	Owns the schema and the layer contract. No migration merges without their review.
D2 INTERFACE	Lakshmi	React, component kit, screens, routing, role switching, states	Owns the component kit. Rejects one-off CSS that should have been a component.
D3 MODELLING	Mohit	Feature builder, model ladder, reciprocal scoring, re-ranking, metrics	Owns <code>FEATURE_VERSION</code> and the experiment log. Guards against leakage.
D4 LANGUAGE	Mayank	LLM adapter, brief parsing, embeddings, RAG, explanations, orchestration	Owns every prompt version and the grounding rules. Nothing ungrounded ships.
D5 DATA & DELIVERY	Manjusha	Adapters, OAuth, seed data, integration tests, Docker, CI, deployment	Owns the normalised social contract and the demo dataset. Keeps main deployable.

The name mapping carries over from the execution plan and is provisional — it is inferred from the repository's commit history. Swap it in weekend 1 if the team disagrees; the rotation works with any assignment because it depends only on the count.

The rotation

Four rounds, built so that each person visits each of the four domains they do not steward exactly once, and every domain receives exactly one visitor per round. No gaps, no collisions, no scheduling argument.

PERSON home domain	W1 mob	W2 round 1	W3 round 2	W4 round 3	W5 round 4	W6 gap-fill	W7 home
Harshit	D1 PLATFORM	D2 Interface	D3 Model	D4 Language	D5 Data	weakest domain, self-chosen	home domain, freeze
Lakshmi	D2 INTERFACE	D3 Model	D4 Language	D5 Data	D1 Platform		
Mohit	D3 MODEL	D4 Language	D5 Data	D1 Platform	D2 Interface		
Mayank	D4 LANGUAGE	D5 Data	D1 Platform	D2 Interface	D3 Model		
Manjusha	D5 DATA	D1 Platform	D2 Interface	D3 Model	D4 Language		

Every row contains every colour: each person builds in all five domains. Every column contains every colour once: no domain is ever left without a steward and no domain receives two visitors at once. The rule is simply $visit = (home + round) \bmod 5$.

READ THE MATRIX IN ONE LINE

You steward your home domain every single weekend. On weekends 2–5 you additionally spend Saturday building in one other domain, moving one step around the ring each round. Weekend 1 is a shared build with no rotation; weekend 6 you choose your own weakest area; weekend 7 everyone is home and nothing new is written.

The weekend shape under rotation

The Friday–Sunday rhythm from the execution plan is unchanged. What changes is that Saturday belongs to a different domain than Friday and Sunday do — roughly seven of every sixteen hours, or 44% of the project, is spent outside your home domain.

WHEN	HOURS	MODE	WHAT HAPPENS
Friday 18:00	0.5	TEAM	Stand-up. Each person names the one thing they will finish by Sunday in each mode.
Friday 18:30	0.5	HANDOVER	The rotation ritual. Steward walks the visitor through their domain for thirty minutes — strictly time-boxed. Five pairs, in parallel, all done by 19:00.
Friday 19:00	2	HOME	Steward work. Small, independent, finishable. No new architecture on a Friday.
Saturday	7	VISITING	The rotation block. First two hours paired with the steward at one keyboard; remaining five solo, with the steward reachable. One vertical slice, chosen so it is finishable.
Sunday 10:00	3	VISITING	Finish the slice, write its tests, open the PR. Steward reviews it the same morning, not on Monday.
Sunday 14:00	2.5	HOME	Back to your own domain. Integrate what the visitor built. Fix what their PR exposed.
Sunday 17:30	0.25	SOLO	Field notes: ten lines into <code>docs/domains/<domain>/field-notes.md</code> , plus your contribution log.
Sunday 18:00	0.25	TEAM	Merge freeze, then run the weekend gate together on one machine.

How to run the thirty-minute handover

This is the mechanism the whole model rests on, and it only works because it is prepared in advance. In weekend 1 every steward writes a one-page `docs/domains/<domain>/README.md` answering five fixed questions. From weekend 2 the handover is a walkthrough of that page plus the specific slice.

MINUTES	STEWARD DOES
0–5	Open the domain README. Name the three files that matter most and what each is for.
5–12	Run the domain's tests live. A visitor who has seen the tests pass knows what “working” looks like.
12–20	Walk one existing example of the slice pattern end to end — the thing the visitor is about to copy.
20–26	Name the traps. “Never commit inside a repository.” “The encoder is a singleton.” “Re-embed after any profile write.” These are the field notes from previous visitors.
26–30	Agree the exact slice and its definition of done. Write it in the issue before you stop talking.

The five questions every domain README answers

1. What is this domain responsible for, in two sentences, and what is explicitly *not* its job?
2. Which three files should a newcomer read first, in what order?
3. How do I run just this domain's tests, and what does a healthy run look like?

- 4. What is the standard shape of a change here — the pattern to copy?
- 5. What are the three ways people have broken this so far?

Weekend by weekend

The *what* lives in the execution plan and the Build Manual — flow IDs below refer to Part 7 of the manual. This section says who does it, in which mode, and what they are learning by doing it.

01 One slice, five pairs of hands

FRI 25 – SUN 27 SEP NO ROTATION

Goal: every person has seen one request travel the entire stack before anyone specialises. This is the highest-value cross-training investment in the project and it is worth spending a whole Saturday on.

SESSION	WHO	WHAT
Fri 3 h	all five	Close the three decisions from Part 1 of the manual (45 min). Then each person writes their domain README — the five questions above. This page is what makes every later handover cheap.
Sat 7 h	all five, one keyboard	Mob the walking skeleton. Build one vertical slice together: <code>POST /auth/register</code> → <code>users</code> + <code>user_roles</code> → JWT → <code>GET /me</code> → a React screen that shows both roles active. Rotate the driver every 25 minutes; everyone types. Nobody researches privately — questions are asked out loud.
Sun 6 h	pairs	Split into pairs and extend: <code>HARSHIT + MANJUSHA</code> migration <code>0002</code> , deps and role guards · <code>LAKSHMI + MAYANK</code> login screen, auth store, role switcher, first four components · <code>MOHIT</code> label rubric v1 and <code>compute_completeness</code> with tests.
SUNDAY GATE		Dual-role register → login → switch mode → 403 from the other mode's endpoint, demonstrated by a person who did not write the auth code . Five domain READMEs committed. <code>docker compose up --build</code> works on two laptops.

02 Profiles, seed data — rotation round 1

FRI 2 – SUN 4 OCT

MID-SEM SAT 3 OCT

Goal: real data in the database and creator profiles working end to end. Rotation starts. **Mid-sem review is Saturday** — present the actual state, and take the two hours it costs out of Friday.

PERSON	HOME THREAD — FRI & SUN PM	SATURDAY VISIT	SKILL GAINED
Harshit	D1 migrations 0003–0005; creator profile CRUD as the reference slice (F2)	D2 builds the rate-card editor screen against his own API	React forms, <code>react-hook-form</code> , seeing his own API from the consumer side
Lakshmi	D2 creator profile wizard, three steps	D3 writes three feature functions in <code>build_pair_features</code> with unit tests	What a feature actually is; why missingness is a signal
Mohit	D3 feature builder skeleton, <code>FEATURE_VERSION</code> , split strategy	D4 writes 20 campaign briefs and the extraction schema draft	Constrained-output schemas; why a parsed field needs a confidence
Mayank	D4 encoder singleton; representation builder offline (F5)	D5 pairs on <code>scripts/seed_data.py</code> — loads the 200 collected creators	The normalised contract; provenance flags; why synthetic must be labelled
Manjusha	D5 shrink <code>data/raw/</code> below 40 MB; YouTube adapter behind the contract	D1 writes the preferences upsert slice (F3) — migration to endpoint	SQLAlchemy 2, Alembic, the service/repository split

SUNDAY GATE

200 real creators and 20 campaigns seeded in one command. A creator profile with a rate card created through the UI. Clean clone under 60 MB. **Rotation check:** every Saturday PR was reviewed and merged by its steward before 18:00.

03 Eligibility and the first ranked list – round 2

FRI 9 – SUN 11 OCT

Goal: a brand clicks Generate and gets ranked creators with stored components and stored exclusion reasons. Rule-based scoring is fine – the endpoint shape will not change when the model arrives.

PERSON	HOME	THREAD	SATURDAY	VISIT	SKILL	GAINED
Harshit	D1	migrations 0006–0007; eligibility service R1–R8; the discovery pipeline shell	D3	implements P@K and NDCG@K with hand-computed tests		Ranking metrics from the inside – why NDCG is not accuracy
Lakshmi	D2	discover grid, score bars, freshness stamps	D4	content clustering → creator_content_clusters		Embeddings, KMeans, what a multi-vertical creator vector means
Mohit	D3	E3 semantic-only; the metrics harness; E1 vs E2 vs E3 table	D5	YouTube OAuth round trip, token encryption at rest		OAuth flows, secret handling, why tokens never reach React
Mayank	D4	representation rebuild as a background task, atomic version swap	D1	builds GET /campaigns/{id}/exclusions end to end		Repository patterns, pagination, the common error shape
Manjusha	D5	sync writes snapshots; sponsored-vs-organic classifier; Instagram Graph POC	D2	builds the exclusions audit view		TanStack Query, component composition, empty and error states
SUNDAY		GATE	Recommendations generated in under two seconds; 75 match_scores rows persisted with components and versions; every excluded creator has a row naming its rule. Rotation check: ask each person to explain, in one sentence, what they learned on Saturday. Vague answers mean the slice was too small.			

04 Both directions and the reciprocal score – round 3

Goal: the product becomes genuinely two-sided, and the proposal state machine works from both entry points. This is the weekend where the research claim first becomes demonstrable.

PERSON	HOME	THREAD	SATURDAY	VISIT	SKILL GAINED
Harshit	D1	migration 0008; inverse eligibility (F8); the proposal state machine (F10)	D4	writes the explanation prompt and its numeric validator	Prompt versioning; how hallucination is caught mechanically, not by trust
Lakshmi	D2	the opportunity feed — the creator-side surface that does not exist in the wireframes	D5	integration tests covering both discovery directions	pytest fixtures, seeded-database testing, what an integration test proves
Mohit	D3	creator-side utility; <code>reciprocal()</code> with all four methods; first E5 run	D1	builds <code>POST /proposals/{id}/counter</code> with its version append	Transaction boundaries; why commercial history is never updated in place
Mayank	D4	LLM adapter, structured extraction, brief parser v1 (F6)	D2	builds both proposal composers from wireframe 07	Controlled forms, optimistic updates, role-aware rendering
Manjusha	D5	Instagram adapter to permitted depth; gold-set outreach; seed toward 340	D3	implements mutual NDCG@5 and the one-sided mismatch rate	Two-sided evaluation; why a brand-only metric hides the failure mode
SUNDAY		GATE			
The convergence demo: a brand finds a creator, and that creator independently finds that campaign in their own feed. One proposal, one counter, one accept, one collaboration row. Rotation check — the important one: every person can now name which table their Saturday work wrote to. If anyone cannot, the model is producing tourists and the next round needs bigger slices.					

05 Trained model, explanations, RAG — round 4

Goal: the headline experimental result exists on held-out data, and the AI layer becomes visible in the product. Last rotation round — after this everyone has touched all five domains.

PERSON	HOME	THREAD	SATURDAY	VISIT	SKILL GAINED
Harshit	D1	document upload + background ingest; model loading from <code>model_versions</code> ; performance pass	D5	Docker smoke test in CI; cloud deployment dry run	CI pipelines, container builds, what breaks between laptop and cloud
Lakshmi	D2	confirm-requirements screen; the “Why this match?” panel	D1	builds <code>POST /campaigns/{id}/knowledge/query</code> end to end	Background tasks, file handling, owner-scoped queries
Mohit	D3	train LR / RF / XGBoost; register versions; E4 vs E5 held-out — the result	D2	builds the document upload and policy Q&A screen	Async UI states; rendering a cited source; degraded banners
Mayank	D4	explanation generator; RAG ingest and retriever; the refusal path	D3	runs the error analysis on E5's losses against E4	Reading a confusion pattern; where a reciprocal score costs relevance
Manjusha	D5	end-to-end suite for both traces; prompt-injection fixture; gold data loaded	D4	builds the RAG evaluation set — 30 questions with known answers	Retrieval metrics; groundedness; why a confident unsourced answer is a bug

SUNDAY GATE A results table for E1–E5 on held-out data. Zero hallucinated figures across 20 sampled explanations. A RAG answer citing a page, and a question correctly refused. **Rotation complete:** fill in the skills matrix together and use it to choose weekend 6.

06 Gap-fill and hardening

Goal: each person returns to the domain where the skills matrix says they are weakest and does one more substantial slice there — this time without a pairing session, which is the actual test of whether the rotation worked. Meanwhile the system gets hardened.

MODE	RULE
Choosing	On Friday, each person names the domain where they scored themselves lowest in weekend 5 and claims one open slice there. Two people may not pick the same domain — first claim wins, and the second person takes their next-lowest. It resolves in five minutes.
Working	No pairing hour. The steward answers questions but does not sit down. If the visitor can ship a slice unaided in a domain they had never touched six weeks earlier, the model worked and that sentence belongs in the report.
Remaining work	Fairness re-ranking and E6 with its relevance cost · cold-start experiment · collaboration kanban · degraded banners everywhere · offline demo mode · failure drills with every external key removed · deployment live · demo script written and timed once.
SUNDAY GATE	Full demo runs with the LLM key and both social API keys removed, with visible degraded banners. E6 reported with its relevance trade-off. Deployed URL reachable. Every person has shipped unaided in a domain that was not theirs in September.

07 Home, freeze and the story

Goal: everyone back in their home domain. Nothing new is built. Friday 18:00 is feature freeze — after that, only fixes with a failing test attached.

PERSON	HOME DOMAIN, ALL WEEKEND
Harshit D1	Bug triage; final OpenAPI export; ADRs 011–014; the architecture section of the report
Lakshmi D2	Walk every screen against the demo script; keyboard and contrast pass; screenshots; one rehearsal recording as insurance
Mohit D3	Open the held-out test set once. Final E1–E6 numbers; results, limitations and negative findings
Mayank D4	Final extraction, explanation and RAG evaluation on the 40 sealed briefs; the latency and cost table from <code>ai_execution_logs</code>
Manjusha D5	Frozen demo dataset; final run on the deployed instance; two timed team rehearsals including the failure path
SUNDAY GATE	Clean clone, one command, working demo. Final results table. Deck and report draft complete. Each person can explain their home domain in depth, sketch the other four from memory, and name one thing that did not work.

The skills matrix

Fill this in together on the Sunday of weekends 1, 5 and 7. It takes ten minutes, it makes the weekend 6 choice objective rather than a negotiation, and the three snapshots become a genuine learning-outcome exhibit for the report.

LEVEL	MEANS	EVIDENCE
0	Never touched it	—
2	Can build in it with the steward available	One merged PR in that domain
3	Can build in it unaided	One merged PR with no steward questions asked
4	Can teach it and decide in it	Wrote the domain README; reviewed others' PRs there

Target state by weekend 7

SKILL	DOMAIN	HARSHIT	LAKSHMI	MOHIT	MAYANK	MANJUSHA
SQLAlchemy 2 & Alembic	D1	4	3	3	3	3
FastAPI service layering	D1	4	3	3	3	3
Auth, JWT, role guards	D1	4	3	2	2	3
React + TypeScript forms	D2	3	4	3	3	3
Server state & caching	D2	2	4	2	3	3
Feature engineering	D3	3	3	4	3	3
Ranking models & XGBoost	D3	2	2	4	3	2
Ranking evaluation	D3	3	2	4	3	3
Embeddings & pgvector	D4	3	3	3	4	2
LLM structured output	D4	3	2	3	4	2
RAG & grounding	D4	2	3	2	4	3
OAuth & external APIs	D5	2	2	3	3	4
Docker, CI, deployment	D5	3	2	2	2	4
Testing & fixtures	D5	3	3	3	3	4

The target in one line: nobody below 2 anywhere, everybody at 3 or better in at least eight of the fourteen, and each person at 4 in their home domain. That is a defensible claim about what a five-person team learned, and it is measurable rather than asserted.

Rules that keep rotation safe

#	RULE	WHY IT EXISTS
1	Visitors implement; stewards decide. A visitor may write any code in the domain's established pattern. A visitor may not introduce a new pattern, rename a shared concept, change a schema, or add a dependency.	This is the entire defence against architectural drift, and it costs nothing.
2	The steward is a required reviewer on every PR in their domain, and reviews it the same weekend.	A review on Wednesday is a review nobody acts on.
3	Schema changes go through D1 regardless of who wrote them. A visitor may author a migration; Harshit merges it.	One inconsistent migration costs more than a weekend of rotation gains.
4	One slice per visit, agreed in writing before Saturday starts. If it is not finishable in nine hours, it is the wrong slice.	Unfinished visiting work is the worst kind — the author leaves and the steward inherits half a thing.
5	Tests are the handover. A visiting PR without tests does not merge, even if the code is right.	The tests are what let the steward maintain it after the visitor moves on.
6	Field notes are part of the PR, not an afterthought.	Ten lines from the last visitor are worth more to the next one than an hour of the steward's time.
7	If a weekend gate fails, the next weekend's rotation is cancelled. Everyone stays home until the gate passes.	Rotation is the flexible thing in this plan. The gates are not.
8	Nobody visits the domain that is behind. If D3 is a weekend late, the D3 steward keeps their Saturday.	Teaching costs the steward time exactly when they have none.

What it costs, and what you cut to pay for it

Cross-training is not free and pretending otherwise is how the plan fails in week 5. Here is the bill, and the scope that pays it. The Wall entering scope adds roughly 20 hours on top, so the cut list below now has to cover about 80.

COST	CALCULATION	PERSON-HOURS
Handover ritual	30 min × 2 people × 20 handovers	20
Pairing overhead	Steward's 2 paired hours × 5 × 4 rounds, roughly half of it recovered as review time not spent later	20
Visitor ramp-up	140 visiting hours at roughly 75% effective productivity	35
Field notes & matrix	15 min × 5 × 7 weekends, minus what it saves on the contribution log	5
Total	out of ~560 person-hours	~80 (14%)
Recovered	Weekend 1's mob day removes most week-2 and week-3 onboarding friction; five people who can review any PR cut review latency for the whole project	~20
Net	—	~60 (11%)

The cut list, in cut order

Take these from the top until 60 hours are paid. Every item is a stretch feature — the spine from the Build Manual and all six ranking experiments survive intact, which is what the marks actually depend on.

ORDER	CUT	HOURS	WHAT YOU SAY INSTEAD
1	Agreement drafting and dual approval (F10 step 5–6)	14	Designed and specified; the grounding mechanism is demonstrated by the explanation feature instead
2	Collaboration messaging	6	Out of MVP scope by decision, not by accident — the plan documents already say messaging is minimal
3	Wall reactions, follows and the topic filter — ship posts, comments and the thread→proposal bridge only	10	The community surface exists and converts to work; discovery of <i>people</i> stays manual for now
4	Instagram beyond the Graph POC	10	Two working adapters prove the integration design; the third is blocked on app review, which is true anyway
5	Admin experiment dashboard	6	The results table is generated into the report from <code>experiment_runs</code> — the data exists, the screen was decoration
6	Counter-offer diff UI	4	Versions are listed rather than diffed
7	LangGraph → deterministic orchestration (F12)	16	Only if still short. The graph is designed, scoped and specified; the tools it would sequence all exist and are individually tested

WHY THIS TRADE IS WORTH MAKING

Sixty hours buys five engineers who can each build across the whole stack, review any pull request in the repository, and answer a question about any part of the system in the viva. It costs six stretch features that no examiner will ask about. For a course called *AI/ML in Practice*, the first outcome is closer to the point of the exercise than the second.

When to stop rotating

Have this conversation on the Sunday of weekend 3 and again in weekend 5, deliberately, before the signals become obvious. Abandoning rotation is a legitimate decision; drifting out of it silently is not.

SIGNAL	WHAT IT MEANS	DO THIS
Two gates failed in a row	The plan is over capacity, and rotation is the cheapest thing to remove	Cancel rotation for weekends 5 and 6. Keep weekend 1's mob knowledge and the field notes. Say so in the report as a measured decision.
Visiting PRs consistently need rework	The slices are too large, or the domain README is too thin	Halve the slice size and rewrite that domain's README before cancelling anything. This is usually a briefing problem, not a people problem.
A steward spends Saturday teaching instead of building	The two-hour pairing cap is not being kept	Enforce the cap. After two hours the visitor works from the README and the tests, and asks asynchronously.
Someone rates themselves 0 in a domain after visiting it	They watched rather than built	Give them the keyboard for the whole of their next visit and give the steward a different task in the same room.
Merge conflicts rising sharply	Two people are editing the same files from different modes	Sequence the slices so a visitor and a steward are never in the same module on the same weekend. This is a Friday planning fix.
It is weekend 7	—	Rotation is already over by design. Everyone is home.

What this gives you in the viva

The individual examination is where this model pays off, and it pays off only if each person can do both halves.

Depth — from stewardship

“I own the modelling domain. I chose harmonic mean over the arithmetic and minimum alternatives, here is the comparison on held-out data, and here is the case where it costs us relevance.” That answer needs seven weeks of continuous ownership, which is exactly what stewardship preserves.

Breadth — from rotation

“I also built the OAuth callback and the rate-card screen, so I can tell you why the token never reaches React and why the rate card is replace-as-a-set rather than row-by-row.” That answer needs to have written the code, not to have read about it.

Judgement — from the field notes

Seven weekends of ten-line entries give each person a concrete record of what surprised them and what they got wrong. “What did not work?” is the question that separates answers, and it cannot be improvised.

The team-level claim

Three dated snapshots of the skills matrix turn “we all learned a lot” into a measured outcome with a method behind it. Put the weekend 1 and weekend 7 matrices side by side in the report appendix.

Everyone Builds Everything · staffing model for the BrandBridge AI capstone · companion to the Build Manual and the seven-weekend execution plan · compiled 20 September 2026.

The rotation is $\text{visit} = (\text{home} + \text{round}) \bmod 5$ over four rounds, which gives every person all four non-home domains exactly once and every domain exactly one visitor per round. It works for any five-person assignment, so swap the names freely in weekend 1.